МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)
Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №17 по курсу «Информационный поиск»

Автотесты и модульное тестирование

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 08.11.2026
Оценка:

Москва, 2026

Содержание

1 Постановка задачи **3**

2 Метод решения **3**

3 План тестирования **4**

4 Результаты **4**

4.1 Запуск тестового плана 4

4.2 Воспроизводимость 5

5 Вывод **6**

1 Постановка задачи

Подготовить воспроизводимый набор автоматических проверок для компонентов учебной поисковой системы. Обосновать тестовый план, покрывающий основные правила обработки документов, построения индекса и выполнения запросов.

2 Метод решения

Основные C++-компоненты проверяются GoogleTest. Тесты токенизатора вызывают его непосредственно; индекс и поиск проверяются на небольших синтетических корпусах с заранее известными результатами. Тесты Python unittest проверяют правила робота, HTML-парсеры, экспорт, веб-обвязку и обработку ошибок. Модульные тесты используют небольшие входы с известным результатом. Интеграционные тесты запускаются с отдельной временной MongoDB.

Метаданные фильтров хранятся отдельно от текстового индекса. Для каждого ID сохраняются сайт, тип и год публикации. Год извлекается из даты статьи; если дата отсутствует или текущая запись MongoDB отличается от снимка, год остаётся неизвестным. Поиск применяет фильтр до ограничения выдачи, поэтому общее число результатов и top-k считаются по отобранным документам.

Общий запуск выполняет `run_tests.py --lab all`: он пересобирает C++-тесты в Docker, выполняет CTest по отдельным именованным случаям, после чего запускает Python unittest в подробном режиме. Скрипт `run_lab17.py` сохраняет полный стандартный вывод и ошибки в `lab17/tests.txt`, запись о тестовом образе в `lab17/build.txt`, журнал временной MongoDB в `lab17/mongo.txt` и сводку с контрольными суммами в `lab17/summary.json`. Если образа нет, он собирается из Dockerfile; если есть, используется существующий. C++-программы при каждом прогоне пересобираются из текущих исходников. Тестовая MongoDB запускается в отдельном контейнере без постоянного тома и останавливается после прогона.

Для Variable Byte использован отдельный эталонный кодировщик, реализованный в тесте по определению формата. На граничных значениях 0, 127, 128, 16383, 16384, 2^{32}, 2^{63} и $2^{64} - 1$ проверяется восстановление числа из файла и из памяти. Декодер должен остановиться ровно после текущего числа и не поглотить начало следующего. Каждый неполный префикс десятибайтовой записи максимального числа должен быть отвергнут.

Для токенизатора проверено свойство объединения независимых полей. Если между двумя корректными полями вставить разделитель, последовательности терминов должны соединиться без изменений; позиции и байтовые границы сдвигаются на известное число токенов и байтов. В проверке есть русские слова, игровые имена, версия и C++/C#.

Тесты Python проверяют правила обработки URL и метаданных. Канонизация удаляет рекламные параметры, сохраняя значимую пагинацию, и повторный вызов не меняет URL. Перенаправление StopGame применяется только к числовому ID новости. Политика исключения отличает страницу категории от вложенной публикации. В JSON-LD находится вложенная публикация; повреждённый блок не скрывает следующий корректный.

3 План тестирования

План составлен по поведению компонентов и возможным отказам. Для каждой группы указан независимый ожидаемый результат, а не только отсутствие аварийного завершения.

Компонент	Проверяемое свойство	Ожидаемый результат
Токенизатор	UTF-8, игровые имена, границы, соединение полей	Точные токены и байтовые смещения; ошибка на повреждённом UTF-8
Лемматизатор	Формы, неоднозначность, неизвестные слова	Все словарные варианты без выдуманных лемм
Индекс	Постинги, координаты, пустые документы и большие ID	Уникальные возрастающие списки, целостный формат
Поиск	AND/OR/NOT, фразы, порядок и соседство	Совпадение с таблицей истинности и независимой проверкой фразы
Сжатие	VB и контрольные точки	Точное восстановление; отказ на усечении и чужом индексе
Ранжирование	TF-IDF, нормировка, top-k	Совпадение с независимой формулой и полной сортировкой
Фильтры	Сайт, тип, год; неизвестная дата; совместный выбор	Точное число совпадений, корректный top-k и сохранение выбора в форме
Робот	URL-категории, метаданные и ошибки	Статьи сохраняются, каталоги отсеиваются, неверные данные не принимаются
Интерфейсы	CLI и веб-форма	UTF-8 и корректный ответ после ошибочного запроса

В наборе использованы фикстуры реальных публикаций для токенизатора, небольшой словарь для морфологии, таблицы истинности для булева поиска, собственное скользящее окно для проверки фраз и прямой расчёт косинуса для TF-IDF. Это позволяет находить ошибки в разных частях системы, не сравнивая программу только с её же повторным запуском.

4 Результаты

4.1 Запуск тестового плана

Команда `python run_lab17.py` выполнена из корня проекта. Готовый Docker-образ использован повторно; исходники C++ собраны заново. Тестовая MongoDB запущена отдельно и остановлена после проверок.

Набор	Всего	Успешно	Пропущено
CTest: GoogleTest и проверки CLI	154	154	0
Python unittest	296	296	0

Все проверки завершились без ошибок. Шесть интеграционных тестов работали с временной MongoDB; каждый использовал отдельную базу со случайным именем. Журнал содержит 154 отдельных строки CTest с именами проверок и результатами, а также имена и результаты 296 Python-тестов. Сводка сохраняет результаты и контрольные суммы файлов запуска.

Случаи выявляют определённые классы регрессий. Например, неверная обработка границы 127/128 или усечённого VB-кода нарушит тест декодера; потеря байтовой границы после соединения полей нарушит свойство токенизатора. Ошибочное обобщение правила каталога PC Gamer приведёт к отбрасыванию вложенной статьи, что покажет тест URL-политики. Тест фильтров проверяет совместный выбор сайта, типа и года на маленьком индексе: первый результат находится после неподходящих документов, а число совпадений не зависит от лимита выдачи. Проверка веб-формы передаёт все три значения в поиск и сохраняет выбранные пункты. Проверка каталога подтверждает, что год загрузки не подменяет отсутствующую дату статьи и что дата изменившегося документа не переносится в старый снимок.

Малые фикстуры не воспроизводят все сбои большого корпуса и внешних сайтов. Однако они позволяют точно проверить поведение компонентов на известных входах.

4.2 Воспроизводимость

Исходники тестов находятся в `../tests/cpp/` и `../tests/python/`. Команды и тестовый план приведены в `lab17_run.md`. Журналы и сводка находятся в `lab17/`. Для полной проверки используется `python run_lab17.py`.

Исходный код: github.com/kirbasss/MAI-IR/tree/main.

Рис. 1 — Ссылка на исходный код и тесты

5 Вывод

В этой работе я собрал единый проверяемый план автотестов поисковой системы. Тесты охватывают главные алгоритмы и пограничные случаи кодирования чисел, токенизации, обработки URL и чтения метаданных. Интеграционные проверки робота выполнены на отдельной временной MongoDB. Полный прогон завершился успешно: 154 проверки CTest и 296 Python-тестов.

Теперь результат можно повторить одной командой и проверить по именованному журналу. Малый набор фикстур не отражает все состояния большого корпуса; по обнаруженным сбоям следует добавлять новые случаи.